

19



Architecture, Implementation, and Testing

You don't make progress by standing on the sidelines, whimpering and complaining. You make progress by implementing ideas.

—Shirley Hufstедler

Although this is a book about software architecture—you've noticed that by now, no doubt—we need to remind ourselves from time to time that architecture is not a goal unto itself, but only the means to an end. Building systems from the architecture is the end game, systems that have the qualities necessary to meet the concerns of their stakeholders.

This chapter covers two critical areas in system-building—implementation and testing—from the point of view of architecture. What is the relationship of architecture to implementation (and vice versa)? What is the relationship of architecture to testing (and vice versa)?

19.1 Architecture and Implementation

Architecture is intended to serve as the blueprint for implementation. The sidebar “Potayto, Potahto . . .” makes the point that architectures and implementations rely on different sets of vocabulary, which results in development tools usually serving one community or the other fairly well, but not both. Frequently the implementers are so engrossed in their immediate task at hand that they make

implementation choices that degrade the modular structure of the architecture, for example.

This leads to one of the most frustrating situations for architects. It is very easy for code and its intended architecture to drift apart; this is sometimes called “architecture erosion.” This section talks about four techniques to help keep the code and the architecture consistent.

Embedding the Design in the Code

A key task for implementers is to faithfully execute the prescriptions of the architecture. George Fairbanks, in *Just Enough Architecture*, prescribes using an “architecturally-evident coding style.” Throughout the code, implementers can document the architectural concept or guidance that they’re reifying. That is, they can “embed” the architecture in their implementations. They can also try to localize the implementation of each architectural element, as opposed to scattering it across different implementation entities.

This practice is made easier if implementers (consistently across a project) adopt a set of conventions for how architectural concepts “show up” in code. For example, identifying the layer to which a code unit belongs will make it more likely that implementers and maintainers will respect (and hence not violate) the layering.

Frameworks

“Framework” is a terribly overused term, but here we mean a reusable set of libraries or classes for a software system. “Library” and “class” are implementation-like terms, but frameworks have broad architectural implications—they are a place where architecture and implementation meet. The classes (in an object-oriented framework) are appropriate to the application domain of the system that is being constructed. Frameworks can range from small and straightforward (such as ones that provide a set of standard and commonly used data types to a system) to large and sophisticated. For example, the AUTomotive Open System ARchitecture (AUTOSAR) is a framework for automotive software, jointly developed by automobile manufacturers, suppliers, and tool developers.

Frameworks that are large and sophisticated often encode architectural interaction mechanisms, by encoding how the classes (and the objects derived from them) communicate and synchronize with each other. For example, AUTOSAR is an architecture and not (just) an architecture framework.

A framework amounts to a substantial (in some cases, enormous) piece of reusable software, and it brings with it all of the advantages of reuse: saving time and cost, avoiding a costly design task, encoding domain knowledge, and decreasing the chance of errors from individual implementers coding the same thing differently and erroneously. On the other hand, frameworks are difficult to

design and get correct. Adopting a framework means investing in a selection process as well as training, and the framework may not provide all the functionality that you require. The learning curve for a framework is often extremely steep. A framework that provides a complete set of functionality for implementing an application in a particular domain is called a “platform.”

Code Templates

A template provides a structure within which some architecture-specific functionality is achieved, in a consistent fashion system-wide. Many code generators, such as user interface builders, produce a template into which a developer inserts code, although templates can also be provided by the development environment.

Suppose that an architecture for a high-availability system prescribes that every component that implements a critical responsibility must use a failover technique that switches control to a backup copy of itself in case a fault is detected in its operation.

The architecture could, and no doubt would, describe the failover protocol. It might go something like this:

In the event that a failure is detected in a critical-application component, a switchover occurs as follows:

1. A secondary copy, executing in parallel in background on a different processor, is promoted to the new primary.
2. The new primary reconstitutes with the application’s clients by sending them a message that means, essentially: The operational unit that was serving you has had a failure. Were you waiting for anything from us at the time? It then proceeds to service any requests received in response.
3. A new secondary is started to serve as a backup for the new primary.
4. The newly started secondary announces itself to the new primary, which starts sending it messages as appropriate to keep it up to date while it is executing in background.

If failure is detected within a secondary, a new one is started on some other processor. It coordinates with its primary and starts receiving state data.

Even though the primary and secondary copies are never doing the same thing at the same time (the primary is performing its duty and sending state updates to its backups, and the secondaries are waiting to leap into action and accepting state updates), both components come from identical copies of the same source code.

To accomplish this, the coders of each critical component would be expected to implement that protocol. However, a cleverer way is to give the coder a code template that contains the tricky failover part as boilerplate and contains fill-in-the-blank sections where coders can fill in the implementation for the functionality that is unique to each application. This template could be embedded in

the development environment so that when the developer specifies that the module being developed is to support a failover protocol, the template appears as the initial code for the module.

An example of such a template, taken from an air traffic control system, is illustrated in Figure 19.1. The structure is a continuous loop that services incoming events. If the event is one that causes the application to take a normal (non-fault-tolerance-related) action, it carries out the appropriate action, followed by an update of its backup counterparts' data so that the counterpart can take over if necessary. Most applications spend most of their time processing normal events. Other events that may be received involve the transfer (transmission and reception) of state and data updates. Finally, there is a set of events that involves both the announcement that this unit has become the primary and requests from clients for services that the former (now failed) primary did not complete.

Using a template has architectural implications: it makes it simple to add new applications to the system with a minimum of concern for the actual workings of the fault-tolerant mechanisms designed into the approach. Coders and maintainers of applications do not need to know about message-handling mechanisms except abstractly, and they do not need to ensure that their applications are fault tolerant—that has been handled architecturally.

Code templates have implications for reliability: once the template is debugged, then entire classes of coding errors across the entire system disappear. But in the context of this discussion, templates represent a true common ground where the architecture and the implementation come together in a consistent and useful fashion.

Keeping Code and Architecture Consistent

Code can drift away from architecture in a depressingly large number of ways. First, there may be no constraints imposed on the coders to follow the architecture. This makes no apparent sense, for why would we bother to invest in an architecture if we aren't going to use it to constrain the code? However, this happens more often than you might think. Second, some projects use the published architecture to start out, but when problems are encountered (either technical or schedule-related), the architecture is abandoned and coders scramble to field the system as best they can. Third (and perhaps most common), after the system has been fielded, changes to it are accomplished with code changes only, but these changes affect the architecture. However, the published architecture is not updated to guide the changes, nor updated afterward to keep up with them.

One simple method to remedy the lack of updating the architecture is to not treat the published architecture as an all-or-nothing affair—it's either all correct or all useless. Parts of the architecture may become out of date, but it will help enormously if those parts are marked as “no longer applicable” or “to be revised.” Conscientiously marking sections as out of date keeps the architecture

documentation a living document and (paradoxically) sends a stronger message about the remainder: it is still correct and can still be trusted.

```

terminate:= false
initialize application/application protocols
ask for current state (image request)
Loop
Get_event
Case Event_Type is
-- "normal" (non-fault-tolerant-related) requests to
-- perform actions; only happens if this unit is the
-- current primary address space
when X => Process X
Send state data updates to other address spaces
when Y => Process Y
Send state data updates to other address spaces
...
when Terminate_Directive => clean up resources; terminate
:= true
when State_Data_Update => apply to state data
-- will only happen if this unit is a secondary address
-- space, receiving the update from the primary after it
-- has completed a "normal" action sending, receiving
-- state data
when Image_Request => send current state data to new
address space
when State_Data_Image => Initialize state data
when Switch_Directive => notify service packages of
change in rank
-- these are requests that come in after a PAS/SAS
-- switchover; they report services that they had
-- requested from the old (failed) PAS which this unit
-- (now the PAS) must complete. A, B, etc. are the names
-- of the clients.
when Recon_from_A => reconstitute A
when Recon_from_B => reconstitute B
...
when others => log error
end case
exit when terminate
end loop

```

FIGURE 19.1 A code template for a failover protocol. "Process X" and "Process Y" are placeholders for application-specific code.

In addition, strong management and process discipline will help prevent erosion. One way is to mandate that changes to the system, no matter when they occur, are vetted through the architecture first. The alternatives for achieving code alignment with the architecture include the following:

- *Sync at life-cycle milestone.* Developers change the code until the end of some phase, such as a release or end of an iteration. At that point, when the schedule pressure is less, the architecture is updated.
- *Sync at crisis.* This undesirable approach happens when a project has found itself in a technical quagmire and needs architectural guidance to get itself going again.
- *Sync at check-in.* Rules for the architecture are codified and used to vet any check-in. When a change to the code “breaks” the architecture rules, key project stakeholders are informed and then either the code or the architecture rules must be modified. This process is typically automated by tools.

These alternatives can work only if the implementation follows the architecture mostly, departing from it only here and there and in small ways. That is, it works when syncing the architecture involves an update and not a wholesale overhaul or do-over.

Potayto, Potahto, Tomayto, Tomahto— Let’s Call the Whole Thing Off!

One of the most vexing realities about architecture-based software development is the gulf between architectural and implementation *ontologies*, the set of concepts and terms inherent in an area. Ask an architect what concepts they work with all day, and you’re likely to hear things like modules, components, connectors, stakeholders, evaluation, analysis, documentation, views, modeling, quality attributes, business goals, and technology roadmaps.

Ask an implementer the same question, and you likely won’t hear any of those words. Instead you’ll hear about objects, methods, algorithms, data structures, variables, debugging, statements, code comments, compilers, generics, operator overloading, pointers, and build scripts.

This is a gap in language that reflects a gap in concepts. This gap is, in turn, reflected in the languages of the tools that each community uses. UML started out as a way to model object-oriented designs that could be quickly converted to code—that is, UML is conceptually “close” to code. Today it is a *de facto* architecture description language, and likely the most popular one. But it has no built-in concept for the most ubiquitous of architectural concepts, the layer. If you want to represent layers in UML, you have to adopt some convention to do it. Packages stereotyped as <<layer>>, associated with stereotyped <<allowed to use>> dependencies do the trick. But it *is* a trick, a workaround for a language deficiency. UML has “connectors,” two of

them in fact. But they are a far cry from what architects think of as connectors. Architectural connectors can and do have rich functionality. For instance, an enterprise service bus (ESB) in a service-oriented architecture handles routing, data and format transformation, technology adaptation, and a host of other work. It is most natural to depict the ESB as a connector tying together services that interact with each other through it. But UML connectors are impoverished things, little more than bookkeeping mechanisms that have no functionality whatsoever. The delegation connector in UML exists merely to associate the ports of a parent component with ports of its nested children, to send inputs from the outside into a child's input port, and outputs from a child to the output port of the parent. And the assembly connector simply ties together one component's "requires" interface with another's "provides" interface. These are no more than bits of string to tie two components together. To represent a true architectural connector in UML, you have to adopt a convention—another workaround—such as using simple associations tagged with explanatory annotations, or abandon the architectural concept completely and capture the functionality in another component.

Part of the concept gap between architecture and implementation is inevitable. Architectures, after all, are abstractions of systems and their implementations. Back in Chapter 2, we said that was one of the valuable properties of architecture: you could build many different systems from one. And that's what an abstraction is: a one-to-many mapping. One abstraction, many instances; one architecture, many implementations. That architecture is an abstraction of implementation is almost its whole point: architecture lets us achieve intellectual control over a system without having to capture, let alone master, all of the countless and myriad truths about its implementation.

And here comes the gap again: All of those truths about its implementation are what coders produce for a living, without which the system remains but an idea. Architects, on the other hand, dismiss all of that reality by announcing that they are not interested in implementation "details."

Can't we all get along?

We could. There is nothing inherently impossible about a language that embraces architectural as well as coding concepts, and several people have proposed some. But UML is beastly difficult to change, and programming language purveyors all seem to focus their attention down on the underlying machine and not up to the architecture that is directing the implementation.

Until this gap is resolved, until architects and coders (and their tools) speak the same conceptual language, we are likely to continue to deal with the most vexing result of this most vexing reality: writing code (or introducing a code change) that ignores the architecture is the easiest thing in the world.

The good news is that even though architecture and implementation speak different languages, they aren't languages from different planets. Concepts in one ontology usually correspond pretty well to concepts in another. Frameworks are an area where the languages enjoy a fair amount of overlap. So are interfaces. These constructs live on the cusp of the two domains, and provide hope that we might one day speak the same language.

—PCC

19.2 Architecture and Testing

What is the relationship between architecture and testing? One possible answer is “None,” or “Not much.” Testing can be seen as the process of making sure that a software system meets its requirements, that it brings the necessary functionality (endowed with the necessary quality attributes) to its user community. Testing, seen this way, is simply connected to requirements, and hardly connected to architecture at all. As long as the system works as expected, who cares what the architecture is? Yes, the architecture played the leading role in *getting* the system to work as expected, thank you very much, but once it has played that role it should make a graceful exit off the stage. Testers work with requirements: Thanks, architecture, but we’ll take it from here.

Not surprisingly, we don’t like that answer. This is an impoverished view of testing, and in fact an unrealistic one as well. As we’ll see, architecture *cannot help* but play an important role in testing. Beyond that, though, we’ll see that architecture can help make testing less costly and more effective when embraced in testing activities. We’ll also see what architects can do to help testers, and what testers can do to take advantage of the architecture.

Levels of Testing and How Architecture Plays a Role in Each

There are “levels” of testing, which range from testing small, individual pieces in isolation to an entire system.

- *Unit testing* refers to tests run on specific pieces of software. Unit testing is usually a part of the job of implementing those pieces. In fact, unit tests are typically written by developers themselves. When the tests are written before developing the unit, this practice is known as test-driven development.

Certifying that a unit has passed its unit tests is a precondition for delivery of that unit to integration activities. Unit tests test the software in a standalone fashion, often relying on “stubs” to play the role of other units with which the tested unit interacts, as those other units may not yet be available. Unit tests won’t usually catch errors dealing with the interaction between elements—that comes later—but unit tests provide confidence that each of the system’s building blocks is exhibiting as much correctness as is possible on its own.

A unit corresponds to an architectural element in one of the architecture’s module views. In object-oriented software, a unit might correspond to a class. In a layered system, a unit might correspond to a layer, or a part of a layer. Most often a unit corresponds to an element at the leaf of a module decomposition tree.

Architecture plays a strong role in unit testing. First, it defines the units: they are architectural elements in one or more of the module views. Second, it defines the responsibilities and requirements assigned to each unit.

Modifiability requirements can also be tested at unit test time. How long it will take to make specified changes can be tested, although this is seldom done in practice. If specified changes take too long for the developers to make, imagine how long they will take when a new and separate maintenance group is in charge without the intimate knowledge of the modules.

Although unit testing goes beyond architecture (tests are based on nonarchitectural information such as the unit's internal data structures, algorithms, and control flows), they cannot begin their work without the architecture.

- *Integration testing* tests what happens when separate software units start to work together. Integration testing concentrates on finding problems related to the interfaces between elements in a design. Integration testing is intimately connected to the specific increments or subsets that are planned in a system's development.

The case where only one increment is planned, meaning that integration of the entire system will occur in a single step, is called “big bang integration” and has largely been discredited in favor of integrating many incrementally larger subsets. Incremental integration makes locating errors much easier, because any new error that shows up in an integrated subset is likely to live in whatever new parts were added this time around.

At the end of integration testing, the project has confidence that the pieces of software work together correctly and provide at least some correct system-wide functionality (depending on how big a subset of the system is being integrated). Special cases of integration testing are these:

- System testing, which is a test of all elements of the system, including software and hardware in their intended environment
- Integration testing that involves third-party software

Once again, architecture cannot help but play a strong role in integration testing. First, the increments that will be subject to integration testing must be planned, and this plan will be based on the architecture. The uses view is particularly helpful for this, as it shows what elements must be present for a particular piece of functionality to be fielded. That is, if the project requires that (for example) in the next increment of a social networking system users will be able to manage photographs they've allowed other users to post in their own member spaces, the architect can report that this new functionality is part of the `user_permissions` module, which will use a new part of the `photo_sharing` module, which in turn will use a new structure in the master `user_links` database, and so forth. Project management will know, then, that all of the software must be ready for integration at the same time.

Second, the interfaces between elements are part of the architecture, and those interfaces determine the integration tests that are created and run.

Integration testing is where runtime quality attribute requirements can be tested. Performance and reliability testing can be accomplished. A

sophisticated test harness is useful for performing these types of tests. How long does an end-to-end synchronization of a local database with a global database take? What happens if faults are injected into the system? What happens when a process fails? All of these conditions can be tested at integration time.

Integration testing is also the time to test what happens when the system runs for an extended period. You could monitor resource usage during the testing and look for resources that are consumed but not freed. Does your pool of free database connections decrease over time? Then maybe database connections should be managed more aggressively. Does the thread pool show signs of degradation over time? Ditto.

- *Acceptance testing* is a kind of system testing that is performed by users, often in the setting in which the system will run. Two special cases of acceptance testing are alpha and beta testing. In both of these, users are given free rein to use the system however they like, as opposed to testing that occurs under a preplanned regimen of a specific suite of tests. Alpha testing usually occurs in-house, whereas beta testing makes the system available to a select set of end users under a “User beware” proviso. Systems in beta test are generally quite reliable—after all, the developing organization is highly motivated to make a good first impression on the user community—but users are given fair warning that the system might not be bug-free or (if “bug-free” is too lofty a goal) at least not up to its planned quality level.

Architecture plays less of a role in acceptance testing than at the other levels, but still an important one. Acceptance testing involves stressing the system’s quality attribute behavior by running it at extremely heavy loads, subjecting it to security attacks, depriving it of resources at critical times, and so forth. A crude analogy is that if you want to bring down a house, you can whale away at random walls with a sledgehammer, but your task will be accomplished much more efficiently if you consult the architecture first to find which of the walls is holding up the roof. (The point of testing is, after all, to “bring down the house.”)

Overlaying all of these types of testing is regression testing, which is testing that occurs after a change has been made to the system. The name comes from the desire to uncover old bugs that might resurface after a change, a sign that the software has “regressed” to a less mature state. Regression testing can occur at any of the previously mentioned levels, and often consists of rerunning the bank of tests and checking for the occurrence of old (or for that matter, new) faults.

Black-Box and White-Box Testing

Testing (at any level) can be “black box” or “white box.” Black-box testing treats the software as an opaque “black box,” not using any knowledge about the

internal design, structure, or implementation. The tester's only source of information about the software is its requirements.

Architecture plays a role in black-box testing, because it is often the architecture document where the requirements for a piece of the system are described. An element of the architecture is unlikely to correspond one-to-one with a requirement nicely captured in a requirements document. Rather, when the architect creates an architectural element, he or she usually assigns it an amalgamation of requirements, or partial requirements, to carry out. In addition, the interface to an element also constitutes a set of "requirements" for it—the element must happily accept the specified parameters and produce the specified effect as a result. Testers performing black-box testing on an architectural element (such as a major subsystem) are unlikely to be able to do their jobs using only requirements published in a requirements document. They need the architecture as well, because the architecture will help the tester understand what portions of the requirements relate to the specified subsystem.

White-box testing makes full use of the internal structures, algorithms, and control and data flows of a unit of software. Tests that exercise all control paths of a unit of software are a primary example of white-box testing. White-box testing is most often associated with unit testing, but it has a role at higher levels as well. In integration testing, for example, white-box testing can be used to construct tests that attempt to overload the connection between two components by exploiting knowledge about how a component (for example) manages multiple simultaneous interactions.

Gray-box testing lies, as you would expect, between black and white. Testers get to avail themselves of some, but not all, of the internal structure of a system. For example, they can test the interactions between components but not employ tests based on knowledge of a component's internal data structures.

There are advantages and disadvantages with each kind of testing. Black-box testing is not biased by a design or implementation, and it concentrates on making sure that requirements are met. But it can be inefficient by (for example) running many unit tests that a simple code inspection would reveal to be unnecessary. White-box testing often keys in on critical errors more quickly, but it can suffer from a loss of perspective by concentrating tests to make the implementation break, but not concentrating on the software delivering full functionality under all points in its input space.

Risk-based Testing

Risk-based testing concentrates effort on areas where risk is perceived to be the highest, perhaps because of immature technologies, requirements uncertainty, developer experience gaps, and so forth. Architecture can inform risk-based testing by contributing categories of risks to be considered. Architects can identify areas where architectural decisions (if wrong) would have a widespread impact, where

architectural requirements are uncertain, quality attributes are demanding on the architecture, technology selections risky, or third-party software sources unreliable. Architecturally significant requirements are natural candidates for risk-based test cases. If the architecturally significant requirements are not met, then the system is unacceptable, by definition.

Test Activities

Testing, depending on the project, can consume from 30 to 90 percent of a development's schedule and budget. Any activity that gobbles resources as voraciously as that doesn't just happen, of course, but needs to be planned and carried out purposefully and as efficiently as possible. Here are some of the activities associated with testing:

- *Test planning.* Test activities have to be planned so that appropriate resources can be allocated. "Resources" includes time in the project schedule, labor to run the tests, and technology with which the testing will be carried out. Technology might include test tools, automatic regression testers, test script builders, test beds, test equipment or hardware such as network sniffers, and so forth.
- *Test development.* This is an activity in which the test procedures are written, test cases are chosen, test datasets are created, and test suites are scripted. The tests can be developed either before or after development. Developing the tests prior to development and then developing a module to satisfy the test is a characteristic of test-first development.
- *Test execution.* Here, testers apply the tests to the software and capture and record errors.
- *Test reporting and defect analysis.* Testers report the results of specific tests to developers, and they report overall metrics about the test results to the project's technical management. The analysis might include a judgment about whether the software is ready for release. Defect analysis is done by the development team usually along with the customer, to adjudicate disposition of each discovered fault: fix it now, fix it later, don't worry about it, and so on.
- *Test harness creation.* One of the architect's common responsibilities is to create, along with the architecture, a set of test harnesses through which elements of the architecture may be conveniently tested. Such test harnesses typically permit setting up the environment for the elements to be tested, along with controlling their state and the data flowing into and out of the elements.

Once again, architecture plays a role and informs each of these activities; the architect can contribute useful information and suggestions for each. For test planning, the architecture provides the list of software units and incremental

subsets. The architect can also provide insight as to the complexity or, if the software does not yet exist, the expected complexity of each of the software units. The architect can also suggest useful test technologies that will be compatible with the architecture; for example, Java's ability to support assertions in the code can dramatically increase software testability, and the architect can provide arguments for or against adopting that technology. For test development, the architecture can make it easy to swap datasets in and out of the system. Finally, test reporting and defect analysis are usually reported in architectural terms: *this* element passed all of its tests, but *that* element still has critical errors showing. *This* layer passed the delivery test, but *that* layer didn't. And so forth.

The Architect's Role

Here are some of the things an architect can do to facilitate quality testing. First and foremost, the architect can design the system so that it is highly testable. That is, the system should be designed with the quality attribute of testability in mind. Applying the cardinal rule of architecture ("Know your stakeholders!"), the architect can work with the test team (and, to the extent they have a stake in testing, other stakeholders) to establish what is needed. Together, they can come up with a definition of the testability requirements using scenarios, as described in Chapter 10. Testability requirements are most likely to be a concern of the developing organization and not so much of the customer or users, so don't expect to see many testing requirements in a requirements document. Using those testability requirements, the testability tactics in Chapter 10 can be brought to bear to provide the testability needed.

In addition to designing for testability, the architect can also do these other things to help the test effort:

- Insure that testers have access to the source code, design documents, and the change records.
- Give testers the ability to control and reset the entire dataset that a program stores in a persistent database. Reverting the database to a known state is essential for reproducing bugs or running regression tests. Similarly, loading a test bed into the database is helpful. Even products that don't use databases can benefit from routines to automatically preload a set of test data. One way to achieve this is to design a "persistence layer" so that the whole program is database independent. In this way, the entire database can be swapped out for testing, even using an in-memory database if desired.
- Give testers the ability to install multiple versions of a software product on a single machine. This helps testers compare versions, isolating when a bug was introduced. In distributed applications, this aids testing deployment configurations and product scalability. This capability could require configurable communication ports and provisions for avoiding collisions over resources such as the registry.

As a practical matter, the architect cannot afford to ignore the testing process because if, after delivery, something goes seriously wrong, the architect will be one of the first people brought in to diagnose the problem. In one case we heard about, this involved flying to the remote mountains of Peru to diagnose a problem with mining equipment.

19.3 Summary

Architecture plays a key role in both implementation and testing. In the implementation phase, letting future readers of the code know what architectural constructs are being used, using frameworks, and using code templates all make life easier both at implementation time and during maintenance.

During testing the architecture determines what is being tested at which stage of development. Development quality attributes can be tested during unit test and runtime quality attributes can be tested during integration testing.

Testing, as with other activities in architecture-based development, is a cost/benefit activity. Do not spend as much time testing for faults whose consequences are small and spend the most time testing for faults whose consequences are serious. Do not neglect testing for faults that show up after the system has been executing for an extended period.

19.4 For Further Reading

George Fairbanks gives an excellent treatment of architecture and implementation in Chapter 10 of his book *Just Enough Software Architecture*, which is entitled “The Code Model” [Fairbanks 10].

Mary Shaw long ago recognized the conceptual gap between architecture and implementation and wrote about it eloquently in her article “Procedure Calls Are the Assembly Language of Software Interconnections: Connectors Deserve First-Class Status” [Shaw 94]. In it she pointed out the disparity between rich connectors available in architecture and the impoverished subroutine call that is the mainstay of nearly every programming language.

Details about the AUTOSAR framework can be found at www.autosar.org.

Architecture-based testing is an active field of research. [Bertolino 96b], [Muccini 07], [Muccini 03], [Eickelman 96], [Pettichord 02], and [Binder 94] specifically address designing systems so that they are more testable. In fact, the three bullets concerning the architect’s role in Section 19.2 are drawn from Pettichord’s work.

Voas [Voas 95] defines testability, identifies its contributing factors, and describes how to measure it.

Bertolino extends Voas's work and ties testability to dependability [Bertolino 96].

Finally, Baudry et al. have written an interesting paper that examines the testability of well-known design patterns [Baudry 03].

19.5 Discussion Questions

1. In a distributed system each computer will have its own clock. It is difficult to perfectly synchronize those clocks. How will this complicate making performance measures of distributed systems? How would you go about testing that the performance of a particular system activity is adequate?
2. Plan and implement a modification to a module. Ask your colleagues to do the same modification independently. Now compare your results to those of your colleagues. What is the mean and the standard deviation for the time it takes to make that modification?
3. List some of the reasons why an architecture and a code base inevitably drift apart. What processes and tools might address this gap? What are their costs and benefits?
4. Most user interface frameworks work by capturing events from the user and by establishing callbacks or hooks to application-specific functionality. What limitations do these architectural assumptions impose on the rest of the system?
5. Consider building a test harness for a large system. What quality attributes should this harness exhibit? Create scenarios to concretize each of the quality attributes.
6. Testing requires the presence of a test oracle, which determines the success (or failure) of a test. For scalability reasons, the oracle must be automatic. How can you ensure that your oracle is correct? How do you ensure that its performance will scale appropriately? What process would you use to record and fix faults in the testing infrastructure?
7. In embedded systems faults often occur "in the field" and it is difficult to capture and replicate the state of the system that led to its failure. What architectural mechanisms might you use to solve this problem?
8. In integration testing it is a bad idea to integrate everything all at once (big bang integration). How would you use architecture to help you plan integration increments?

This page intentionally left blank